WEB DEVELOPMENT

Learning Groups: P107, P108, E101, P102

Lecturer: Mr. Maina Samuel

Lesson 6 **learning topics**

- **6.0 java script data types**
- **6.1 JavaScript operator**
- **6.2 java Script if, Else, IF else if, switch statements**
- **6.3 java Script Loops**
- **6.4 java Script Arrays**

6.0 java script data types

JavaScript Data types are used to **identify the type of data that is stored inside a variable during the script execution.**
So JavaScript data types are basically **for identification purposes** to know what is being stored in the variable, when it's stored, not before that.

JavaScript broadly supports three types of Data types; they are:

- Primitive Type
- Reference Type
- Special Data Type

1. JavaScript Primitive Data Type

JavaScript Primitive data types can be classified further into the following types:

i. String Data Type
ii. Boolean Data Type
iii. Number Data Type

i. JavaScript String Data type

Whenever we write a **character or sequence** of characters inside single or double quotes then it becomes String.

```
var str = "Ferari F60”; // string using double quotes
```

```
var str1 = 'Volvo S60';  // string using single quotes.
```

ii. JavaScript Boolean Data type

JavaScript Boolean Data type is used in conditional based programming. It can have two values, either **true** or **false**.

```
var isOn = true; // bulb is on

var isOn = false; // bulb is off
```

iii. JavaScript Number Data type

JavaScript Number Data type can be with or without decimal points and can have negative and positive values.

```
var x = 45; // Number without decimal point

var y = 45.90; // Number with decimal point - floating point

var z = -10; // Number with negative value
```

The JavaScript Number data type also represents some special values like Infinity, -Infinity and Nan. When a positive number is divided by zero (it's a popular case of runtime error), in JavaScript it's represented as Infinity. Similarly, when a negative number is divided by zero we will get -Infinity.

```
var a = 100;

var b = -100;

var c = 0;

alert(a/c);  // Infinity

alert(b/c);  // -Infinity
```

2. JavaScript Composite Data types/ Reference

These data types can hold collections of values and more complex entities. It is further divided into Object, Array and Function.

i. Object data type
ii. Array data type
iii. Function data type

i. Object Data Type

In JavaScript, an object data type is used to store the collection of data. Object's properties are written as **key:value** pairs, which are **separated by commas** and **enclosed within curly braces** {}.

The **key (name) must always be a string**, but the **value can be of any data type**. This is pretty similar to a map data structure in many programming languages which also stores key-value pairs like this.

```
var name = { };    // It will create an empty object.

var emp = {firstname="ram", lastname="singh", salary=20000};
```

ii. Array Type

JavaScript Array data type is written inside a pair of square brackets [] and is used to store multiple values of the same datatype be it strings, numbers etc. The items in a JavaScript array are also written in a comma-separated manner.

Each element in the array gets a numeric position, known as its **index**. The array index starts from **0** or we can say that **array indexes are zero-based**, so that the first array element is arr[0] and not arr[1].

```
// Creating an Array
```

```
var cars = ["Ferrari", "Volvo", "BMW",
"Maseratti"];
```

iii. Function Type

You must be thinking, how a function can be a datatype. But in JavaScript **functions act as a data type which can be assigned to a variable**. JavaScript Function is nothing but **a set of statement inside a code block** which is used to perform a specific operation and **this datatype is of callable in nature**. So, you can call it anywhere in the program whenever needed.

Since functions are objects, so it is possible to assign them to a variable.

Functions can be stored in **variables, objects, and arrays**. Functions can be passed as arguments to other functions too and can be returned from other functions as well.

A function can be helpful in the following scenarios.

- For instance, suppose you want to add some numbers and display the results on a web page. In that case, you can define the code for adding the numbers in a function and call the function whenever needed.
- For repetitive tasks like displaying a message whenever a web page is loaded into the browser, we can use functions.

```
<html>
    <head>
        <script>
            function mySumFunction(a, b)
            {
                // return the result
                return a+b;
            }

            let x = myFunction(8, 7);
            alert(x);
        </script>
    </head>
</html>
```

3. Special Data types

JavaScript also has some special data types, although seeing function called as a data type would have already been special for you. But there are two more.

1. Undefined Data type
2. Null Data type

i. JavaScript Undefined Data Type

When a variable is just declared and is not assigned any value, it has **undefined** as its value. Yes, undefined is a valid data type in JavaScript and it can have only one value which is **undefined**.

```
var a;    // Undefined

alert(a == undefined);    // returns true
```

JavaScript Null Data Type

JavaScript Null data type is used to represent no value. It is not similar to undefined, and neither it is similar to empty value or zero value. The Null datatype means; the variable has been defined but it contains no value.

The Null data type can have only one value, which is **null**.

```
var a = null;

alert(a); // Output will be null
```

6.1 JavaScript operator

Operators are used to performing operations on the operands.

Operators are useful to modify or change data by manipulating the respective variables. For example, there are two variables a and b and we want to **add the values of both of these variables**, to do so we can use arithmetic operator + to perform addition of these two variables.

JavaScript has following types operators:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Conditional Operator (Ternary Operator)
6. Bitwise Operators
7. Miscellaneous Operators
    1. type of
    2. delete
    3. instance of
    4. new
    5. this
    6. in

i. Arithmetic operators

JavaScript arithmetic operator take operand (as a values or variable) and return the single value.

Example of Arithmetic operators

We have numeric variable: x = 10, y = 5 and result.

Operator	Description	Example	Results
+	Addition	result = x + y	result = 15
-	Subtraction	result = x - y	result = 5
*	Multiplication	result = x * y	result = 50
/	Division	result = x / y	result = 2
%	Modulus	result = x % y	result = 0
++	Increment	result = x++ result = x result = ++x	result = 10 result = 11 result = 12
--	Decrement	result = x-- result = x result = --x	result = 12 result = 11 result = 10

NB: **pre increment ++X vs post increment X++**

Value of X=10

++X means increment X by one before i.e generally **x=x+1** same as **++X**: **result = 11**

X++ means increment X after this line **result= 10,11…**

```
<title>JavaScript Arithmetic Operators</title>
</head>
<body>
<pre>   <!-- Use pre element for work document.writeln() method -->
<script type="text/javascript">

    var x = 10, y = 5;
    document.writeln("Add : ", x + y);     //  Addition: 15
    document.writeln("Sub : ", x - y);     //  Subtraction: 5
    document.writeln("Mul : ", x * y);     //  Multiplication: 50
    document.writeln("Div : ", x / y);     //  Division: 2
    document.writeln("Mod : ", x % y);     //  Modulus: 0

    document.writeln("-------Increment------");
    document.writeln(x++);       //  x: 10, x become now 11
    document.writeln(x);         //  x: 11
    document.writeln(++x);       //  x become now 12, x: 12

    document.writeln("-------Decrement------");
    document.writeln(x--);       //  x: 12, x become now 11
    document.writeln(x);         //  x: 11
    document.writeln(--x);       //  x become now 10, x: 10

</script>
</pre>
</body>
</html>
```

```
Add : 15
Sub : 5
Mul : 50
Div : 2
Mod : 0
-------Increment------
10
11
12
-------Decrement------
12
11
10
```

ii. JavaScript Comparison Operators

JavaScript comparison operator determine the two operands satisfied the given condition. Comparison operator return either true or false.

Operator	Sign	Description
Equal	==	If both operands are equal, returns true.
Identical equal	===	If both operands are equal and/or same data type, returns true.
Not equal	!=	If both operands are not equal, returns true.
Identical not equal	!==	If both operands are not equal and/or same data type, returns true.
Greater than	>	If left operand larger than right operand, return true.
Less then	<	If left operand smaller than right operand, return true.
Greater than, equal	>=	If left operand larger or equal than right operand, return true.
Less than, equal	<=	If left operand smaller or equal than right operand, return true.

```
<!DOCTYPE html>
<head>
  <title>JavaScript Comparison Operators</title>
</head>
<body>
<pre>   <!-- Use pre element for work document.writeln() method -->
<script type="text/javascript">

    document.writeln("Equal              : ", 4 == 5);        // true
    document.writeln("Equal              : ", 5 == '5');      // true
    document.writeln("Identical equal    : ", 5 === '5');     // false type
      not same

    document.writeln("Not equal          : ", 5 != 10);       // true
    document.writeln("Not equal          : ", 5 != '10');     // true
    document.writeln("Identical not equal: ", 5 !== '10');    // true

    document.writeln("Greater than       : ", 5 > 10);        // false
    document.writeln("Less then          : ", 5 < 10);        // true

    document.writeln("Greater than, equal: ", 5 >= 5);        // true
    document.writeln("Less than, equal   : ", 5 <= 5);        // true

</script>
</pre>
</body>
</html>
```

```
Equal              : false
Equal              : true
Identical equal    : false
Not equal          : true
Not equal          : true
Identical not equal: true
Greater than       : false
Less then          : true
Greater than, equal: true
Less than, equal   : true
```

iii. JavaScript Logical Operators (Boolean Operators)

JavaScript logical operators return boolean result base on operands.

Operator	Sign	Description
Logical AND	&&	If first operand evaluate and return a true, only that evaluate the second operand otherwise skips. Return true if both are must be true, otherwise return false.
Logical OR	||	Evaluate both operands, Return true if either both or any one operand true, Return false if both are false.
Logical NOT	!	Return the inverse of the given value result true become false, and false become true.

```
<!DOCTYPE html>
<head>

  <title>JavaScript Logical Operators</title>
</head>
<body>
<pre>   <!-- Use pre element for work document.writeln() method -->
<script type="text/javascript">

    document.writeln("AND : ", (5 == 5) && (10 == 10));    // Returns true
    document.writeln("AND : ", true && false);             // Returns false

    document.writeln("OR  : ", (5 == 5) || (5 == 10));     // Returns true
    document.writeln("OR  : ", true || false);             // Returns true

    document.writeln("AND : ", 5 && 10);                   // Returns 10
    document.writeln("OR  : ", 5 || 10);                   // Returns 5

    document.writeln("NOT : ", !5);                        // Returns false
    document.writeln("NOT : ", !true);                     // Returns false
    document.writeln("NOT : ", !false);                    // Returns true

</script>
</pre>
</body>
</html>
```

```
AND : true
AND : false
OR  : true
OR  : true
AND : 10
OR  : 5
NOT : false
NOT : false
NOT : true
```

iv . JavaScript Assignment Operators

JavaScript assignment operators assign values to left operand based on right operand. equal (=) operators is used to assign a values.

```
<!DOCTYPE html>
  <head <title>JavaScript Assignment Operators</title></head>
<body>
<pre>   <!-- Use pre element for work document.writeln() method -->
<script type="text/javascript">

    var x = 17, y = 5;
    var result = x; // Assignment to left operand(result) base on right
      operand(y).
    document.writeln("Assignment : ", result);
    document.writeln("Addition : ", result += y);
    document.writeln("Subtraction : ", result -= y);
    document.writeln("Multiplication : ", result *= y);
    document.writeln("Division : ", result /= y);
    document.writeln("Modulus : ", result %= y, "<br />");

    document.writeln("Bitwise AND : ", result &= y);
    result = 2;     // Reassign value
    document.writeln("Bitwise OR : ", result |= y);
    document.writeln("Bitwise XOR : ", result ^= y, "<br />");

    document.writeln("Shift Left : ", result <<= y);
    document.writeln("Shift Right : ", result >>= y);

</script>
</pre>
</body>
```

```
JavaScript Assignment Operators

Assignment : 17
Addition : 22
Subtraction : 17
Multiplication : 85
Division : 17
Modulus : 2

Bitwise AND : 0
Bitwise OR : 7
Bitwise XOR : 2

Shift Left : 64
Shift Right : 2
```

V. JavaScript Conditional Operator (also call Ternary Operator)

JavaScript conditional operator evaluate the first expression(operand), Base on expression result return either second operand or third operand.

```
<!DOCTYPE html>
<head><title>JavaScript Ternary Operator</title></head>
<body>
<pre>
<script type="text/javascript">

    document.write((5== 10) ? "Same value" : "different value");

</script>
</pre>
</body>
</html>
```

```
different value
```

6.2 java script if statement

JavaScript If-else

The **JavaScript if-else statement** is used *to execute the code whether condition is true or false*. There are three forms of if statement in JavaScript.

1. If Statement
2. If else statement
3. if else if statement
    i. if statement

It evaluates the content only if expression is true.

```
<html>
<body>
<script>
var a=20;
if(a>10){
document.write("value of a is greater than 10");
}
</script>
</body>
</html>
```

value of a is greater than 10

Example 2: SIMPLE INPUT TEXT BOX CALCULATOR

How to calculate Two text box values (**function**) in JavaScript

Passing JavaScript function and using **IF** statement condition

```html
<!DOCTYPE html>
<html>
<head>
    <title>How to calculate two text box values in javascript</title>
</head>
<body>
<h1>Calculate Two Fields</h1>
<h1 id= "answer"></h1>

    num 1<input type="text" id="num1">
    <br>
    <br>
    num 2<input type="text" id="num2">
    <br>
    <br>
    <button onclick="calculate()"> Calculate</button>

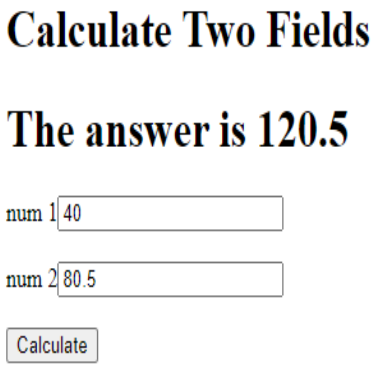
    <script>
        function calculate ()
        {
            var field1=document.getElementById("num1").value;
            var field2=document.getElementById("num2").value;
            var result=parseFloat(field1)+parseFloat(field2);

            if (! isNaN (result))
            {
                document.getElementById("answer").innerHTML="The answer is "+ result;
            }

                }

</script>
</body>
</html>
```

Output



Code Explanation

parse Float function () function is used **to accept the string and convert it into floating –point number.**

if the string does not contain a numeral value or if the first character of the string is not a **number** it returns **NaN**

parse float syntax

parse float (value)

- value this parameter contains a string which is converted to floating-point number.

ii. JavaScript If...else Statement

It evaluates the content whether condition is **true** of **false.**

```
<html>
<body>
<script>
var a=19;
if(a%2==0){
document.write("a is even number");
}
else{
document.write("a is odd number");
}
</script>
</body>
</html>
```

a is odd number

iii. JavaScript If...else if statement

It evaluates the content only if expression is true from several expressions. The signature of JavaScript if else if statement is given below.

```
<html>
<body>
<script>
var a=20;
if(a==10){
document.write("a is equal to 10");
}
else if(a==15){
document.write("a is equal to 15");
}
else if(a==20){
document.write("a is equal to 20");
}
else{
document.write("a is not equal to 10, 15 or 20");
}
</script>
</body>
</html>
```

```
a is equal to 20
```

iv. java script switch statements

The **switch statement** allows to **make a decision from the number of choices**.

If a match is found to a **case label**, the program **executes the associated statement**.

If no match is found with any of the case statements, **only the statements following the default are executed.**

```
<!DOCTYPE html>
<head>
<meta charset=utf-8>
<title>JavaScript Switch statement : Example-1</title>
<link rel="stylesheet" type="text/css" href="example.css">
</head>
<body>
<h1>JavaScript : switch statement</h1>
<form name="form1" action ="#">
Input Grade type : <input type="text" name="text1" value="A" />
<br /><br />
<input type="button" value="Marks check"
onclick="marksgrade()" />
</form>
<script src="switch-statement-example1.js"></script>
</body>
</html>
```

```
function marksgrade()
{
grade = document.form1.text1.value;
switch (grade)
{
case 'A+':
console.log("Marks >= 90");
break;
case 'A':
console.log("Marks [ >= 80 and <90 ]");
break;
case 'B+':
console.log("Marks [ >= 70 and <80 ]");
break;
case 'B':
console.log("Marks [ >= 60 and <70 ]");
break;
case 'C':
console.log("Marks < 60");
break;
default:
console.log("Wrong grade.........");
}
}
```

OUTPUT



6.3 java Script Loops

The **JavaScript loops** are used *to iterate the piece of code* using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array.

1. for loop
2. while loop
3. do-while loop

i. JavaScript for loop

The **JavaScript for loop** *iterates the elements for the fixed number of times*. **It should be used if number of iteration is known.**

Syntax

```
for (initialization; condition; increment)
{
   code to be executed
}
```

```
<!DOCTYPE html>
<html>
<body>
<script>
for (i=1; i<=5; i++)
{
document.write(i + "<br/>")
}
</script>
</body>
</html>
```

```
1
2
3
4
5
```

ii. JavaScript while loop

The **JavaScript while loop** *iterates the elements for the infinite number of times*. It should be **used if number of iteration is not known**.

```
<!DOCTYPE html>
<html>
<body>
<script>
var i=11;
while (i<=15)
{
document.write(i + "<br/>");
i++;
}
</script>
</body>
</html>
```

```
11
12
13
14
15
```

iii. JavaScript do while loop

The **JavaScript do while loop** *iterates the elements for the infinite number of times* like while loop. But, **code is *executed at least* once** whether condition is **true or false.**

Syntax

```
Do
{
   code to be executed
    } while (condition);
```

```
<!DOCTYPE html>
<html>
<body>
<script>
var i=21;
do{
document.write(i + "<br/>");
i++;
}while (i<=25);
</script>
</body>
</html>
```

```
21
22
23
24
25
```

6.4 java Script Arrays

JavaScript array is an object that represents a collection of similar type of elements.

There are 3 ways to construct array in JavaScript

1. By array literal
2. By creating instance of Array directly (using new keyword)
3. By using an Array constructor (using new keyword)

i. By array literal

var array name= [value1, value2....valueN];

values are contained inside [ ] and separated by , (comma).

```
<html>
<body>
<script>
var emp=["Susan","Kamau","mutiso"];
for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br/>");
}
</script>
</body>
</html>
```

Susan
Kamau
mutiso

ii. JavaScript Array directly (new keyword)

new keyword is used to create instance of array.

Syntax

var arrayname=new Array();

```
<html>
<body>
<script>
var i;
var emp = new Array();
emp[0] = "Amit";
emp[1] = "Rose";
emp[2] = "John";

for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br>");
}
</script>
</body>
</html>
```

Amit
Rose
John

iii. By using an Array constructor

Here, you need to **create instance of array** by passing arguments **in constructor** so that we **don't have to provide value explicitly**.

```
<html>
<body>
<script>
var emp=new Array("Jai","Vijay","Smith");
for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br>");
}
</script>
</body>
</html>
```

Jai
Vijay
Smith

Revision Question

1. Write a JavaScript program which compute, the **average marks** of the following students Then, this average is used to determine the corresponding grade.

Student Name	Marks
David	80
Vinoth	77
Divya	88
Ishitha	95
Thomas	68

The grades are computed as follows:

Range	Grade
<60	F
<70	D
<80	C
<90	B
<100	A